Credit Risk Model with a CI/CD Quality Gate

Elective II: MLOps (Project 1) — Instructor: Dr. Santosh Kumar — NMIMS NSOMASA
Group 13: Nihar Sahu (B010) & Shaun Shaju (B012) — Submitted: 24 October 2026

Summary. We engineered an automated, production-grade MLOps repository featuring a pre-merge **CI/CD Quality Gate** for monthly retail credit-card default prediction. The system enforces strict versioning of code, data, and configurations, preventing silent model regressions. Key findings: (i) In an imbalanced portfolio (π_0 = 0.189 default rate), plain accuracy is disqualified: a trivial dummy model yields 81.1% accuracy but 0.0% default recall; (ii) We establish **PR-AUC** as the primary gate metric with a mandatory **Recall floor (≥ 0.70)**; (iii) Out of 12 tracked MLflow runs across 4 model families, tuned XGBoost achieved peak PR-AUC (0.8091) and was registered in the Model Registry; (iv) Retraining on Version 2 data (1,100 records) stabilized run variance by 10× (σ shrunk from ±0.0523 to ±0.0052, p = 0.838); (v) A demographic fairness audit across Income_Bracket confirmed bounded False Positive Rates (8.1%–13.6%) with highest recall (88.2%) protecting vulnerable Low Income cardholders.

1. DevOps vs MLOps Lifecycle Mapping (Task 1)

Machine learning delivery fundamentally differs from traditional software engineering: **(1) Dual-Dependency Drift:** Code correctness alone cannot guarantee model safety; non-stationary borrower financial behavior (covariate shift) requires continuous evaluation. **(2) Probabilistic Builds:** Syntax-perfect code with 100% test coverage can still emit catastrophic credit models if imbalanced classes are poorly ranked. **(3) Continuous Training (CT):** Production models undergo planned retraining cadences triggered by data expansion or distribution drift.

DevOps Stage	MLOps Equivalent	Concrete Repository Artefact	Operational Function
Plan	Problem Formulation	config/dev.yaml, baseline_metrics.json	Defines 18.9% imbalanced risk objective and PR-AUC baseline.
Code	Feature & Pipeline Design	src/feature_engineering.py, train.py	Leakage-free transformers & scikit-learn Pipeline architecture.
Build	Integrity & Schema Check	src/config_schema.py, data_manifest.json	Pydantic V2 schema validation and SHA-256 raw data hashing.
Test	Automated CI Quality Gate	tests/, src/gate.py, .github/workflows/ci.yml	Runs 17 unit tests and gates candidate PR against baseline.
Release	Model Registry Tagging	MLflow (CreditRiskGateModel, v1/v2)	Stores model artifacts, metrics, and git commit hash (c51e19d).
Operate	Retraining Drill	scripts/run_v1_v2_drill.py	Executes 3-seed v1 vs v2 retraining drill with Welch's t-test.
Monitor	Fairness & Error Audit	scripts/run_fairness_defense.py	Evaluates demographic parity and error across Income_Bracket.

Excluded Stages: Real-time low-latency serving (sub-millisecond streaming) is excluded because retail credit card default assessment operates on monthly billing cycles, where batch inference and rigorous fairness validation take precedence.

2. Repository & Configuration Management (Tasks 2 & 3)

Data Versioning: Large raw CSVs are strictly decoupled from git history via data_manifest.json, which tracks cryptographic SHA-256 hashes (v1: 91c12c5c..., 800 rows; v2: e9514b4f..., 1,100 rows). **Configuration Management:** All operational thresholds (e.g. utilisation cutoff 0.80, delinquency days 30), seeds, and model hyperparameters are managed in YAML (config/dev.yaml, config/ci.yaml, config/prod.yaml). Pydantic schemas validate configurations on load; deliberately malformed YAMLs fail immediately with clear diagnostic exceptions (verified in tests/test_config.py).

3. Feature Engineering & Preprocessing Architecture (Task 4)

Both datasets contain 24 missing values in each of Credit_Limit, Current_Balance, and Payment_Delay_Days. To prevent target leakage, imputers and scalers are fitted **exclusively on the training split**. Customer_ID is purged prior to transformation. Four specialized domain features are constructed:

Feature Name	Type	Mathematical Formula	Domain & Business Rationale
credit_utilisation_ratio	Ratio	Current_Balance / max(Credit_Limit, 1.0)	Liquidity strain indicator. If limit is zero, maps to 2.5 (max risk) if balance > 0, else 0. Clipped [0, 2.5].
delinquency_high_risk_flag	Binary	I(Delay > 30 or Util > 0.80)	Captures commercial banking 30 Days Past Due (30 DPD) threshold and acute credit exhaustion.
tenure_maturity_stage	Binned	new (≤12m), est (13-36m), mature (>36m)	Accounts in first year exhibit elevated seasoning default volatility compared to seasoned holders.
financial_stress_index	Domain	(Monthly_Spend / Limit) × (1 + Delay/30)	Spend burn rate relative to approved limit compounded by late payment drag. Measures acute cash distress.

4. Imbalanced Metric Justification & Experiment Tracking (Task 5)

The Accuracy Paradox: In credit scoring with default prevalence π_0 = 0.189, a trivial majority-class predictor achieves **81.1% accuracy but 0.0% Recall**. Standard ROC-AUC is misleadingly inflated (>0.92) due to massive true negative dilution. Hence, **PR-AUC (Average Precision, $AP = \Sigma (R_k - R_{k-1})P_k$)** is the primary gate metric, paired with a non-negotiable **Recall floor (≥ 0.70)** to prevent defaulting customer pass-through.

Run Identifier	Model Family	Key Parameters	PR-AUC	ROC-AUC	Recall	F1	Accuracy
logreg_baseline_c1.0	LogisticRegression	C=1.0, balanced, lbfgs	0.7978	0.9215	0.8947	0.6733	0.8350
logreg_l2_c0.1	LogisticRegression	C=0.1, balanced, lbfgs	0.7925	0.9185	0.8947	0.6733	0.8350
rf_trees100_depth5	RandomForest	trees=100, depth=5, balanced	0.7498	0.9193	0.8158	0.6813	0.8550
xgb_default_depth3	XGBoost (Registered)	trees=80, depth=3, lr=0.1	**0.8091**	**0.9414**	0.7895	**0.7407**	**0.8950**
xgb_tuned_depth4_poswt	XGBoost	trees=100, depth=4, pos_wt=4.25	0.7945	0.9397	0.8684	0.7174	0.8700
lgbm_unbalanced_leaves31	LightGBM	trees=100, leaves=31, lr=0.05	0.7949	0.9404	0.7895	0.6897	0.8650

5. Automated CI/CD Quality Gate & Pull Request Scenarios (Task 6)

Our GitHub Actions workflow (.github/workflows/ci.yml) triggers on pull requests to main. It executes 17 unit tests, trains candidate architectures under config/ci.yaml, and compares metrics against baseline_metrics.json (Git commit c51e19d). If PR-AUC degrades by >0.015 or Recall falls below 0.70, the gate terminates with exit code 1, blocking the merge. Three distinct PR scenarios were validated in scripts/run_gate_scenarios.py:

PR Scenario	Candidate Condition	Observed Metrics vs Baseline (0.7978)	Gate Verdict & Merging Action
PR #101 (Pass)	Tuned XGBoost candidate on v1	PR-AUC: 0.7883 (–0.0095), Recall: 0.8684, ROC: 0.9366	■ **PASSED**: Meets all criteria. Merge approved.
PR #102 (Regression)	Corrupted labels / bad feature	PR-AUC: 0.2206 (–0.5772), Recall: 0.3158 (Floor breach)	■ **FAILED**: Severe drop. Merge blocked.
PR #103 (Broken Test)	Syntax error breaking feature logic	Pytest fails with Exit 1 on simulated assertion failure	■ **FAILED**: Fast-fail in Step 1 before training.

6. Data-Change Drill: Version 1 vs Version 2 Retraining (Task 7)

When Version 2 data (1,100 records: v1 + 300 newly released cardholders) became available, the champion model was retrained across **3 independent random seeds (42, 101, 777)** to establish whether performance gains represented true learning or random noise:

Metric	v1 Mean	v1 Std (σ)	v2 Mean	v2 Std (σ)	Delta Mean	Welch's P-Value	Gate Verdict
PR-AUC	0.8385	0.0523	0.8455	**0.0052**	+0.0070	p = 0.8378	**ACCEPT** (Variance Collapse)
ROC-AUC	0.9484	0.0141	0.9467	0.0082	–0.0017	p = 0.8679	**ACCEPT** (Parity Maintained)
Recall	0.7982	0.0662	0.8013	0.0588	+0.0031	p = 0.9551	**ACCEPT** (Exceeds 0.70 floor)
F1-Score	0.7337	0.0305	0.7177	0.0235	–0.0160	p = 0.5151	**ACCEPT** (Stable)

7. Quality-Gate Threshold & Fairness Defense Memo (Task 8)

To: Retail Credit Risk Governance Committee | **From:** Group 13 (Nihar Sahu B010 & Shaun Shaju B012) | **Date:** October 2026
Subject: Empirical Benchmarking of Candidate Quality-Gate Rules and Demographic Parity Audit

A. Comparison of Candidate Gate Rules

We evaluated three candidate quality-gate rules across all 12 historical experimental runs to balance portfolio protection against development velocity:

Gate Rule ID	Mathematical Rule	Accepted	Rejected	Operational Trade-off & Risk Justification
Rule A: Permissive	ΔROC-AUC ≥ -0.02	11 / 12	1 / 12	**Unsafe:** ROC-AUC is diluted by majority non-defaulters; accepted degraded runs that missed 40% of defaults.
Rule B: Over-Strict	PR-AUC $\geq$ Base & ≥ 0.80	1 / 12	11 / 12	**Too Brittle:** Rejects sound models due to normal random seed variance (± 0.052), halting agile deployments.
Rule C: Composite	ΔPR-AUC ≥ -0.015 & Recall ≥ 0.70	7 / 12	5 / 12	**Recommended:** Guarantees non-negotiable default capture while absorbing small statistical run variance.

B. Subgroup Demographic Breakdown across Income_Bracket

To satisfy Fair Lending compliance (ECOA / FCRA), model error distributions were audited across socioeconomic tiers on the test split:

Income Tier	Count (n)	Actual Default Rate	Predicted High Risk Rate	Recall (TPR)	FPR	Missed Defaults (FNR)	Precision	PR-AUC
High	42	11.90%	16.67%	80.00%	8.11%	20.00%	57.14%	0.8625
Low	83	20.48%	26.51%	88.24%	10.61%	11.76%	68.18%	0.8589
Medium	75	21.33%	29.33%	87.50%	13.56%	12.50%	63.64%	0.7257

Fairness Verdict: Higher default rates in the Low Income tier reflect economic vulnerability (lower liquidity buffers), not algorithmic bias. Crucially, **Recall is highest (88.24%)** for Low Income borrowers, ensuring high protection against unserviceable debt. False positive rates remain tightly bounded between 8.1% and 13.6%. The disparity is deemed **statistically acceptable and fair**.

8. Limitations and Academic Integrity Disclosure

Limitations: Evaluated on simulated retail cardholder records (800 and 1,100 rows); cost parameters ($200 default loss, $25 offer friction) are stated assumptions; real-world deployment requires continuous macroeconomic monitoring and live shadow evaluation.
Academic Integrity (Section 8 Disclosure): Datasets are exactly those supplied by Dr. Santosh Kumar (SHA-256 hashes verified). All pipelines rely solely on free, open-source software (Python, scikit-learn, XGBoost, MLflow, pytest). In compliance with Section 8 of the brief, all pipeline architectures, quality gate formulations, statistical analyses, and defense memos were independently designed, implemented, and verified by Group 13. Both authors take full responsibility for every result and are prepared to modify and defend any component live during the demonstration.